Shiksha Setu: A Measurement-Driven, Local-First AI Tutoring Platform for Multilingual Indian Education

Rachit Ranka
Department of Computer Science
Chandigarh University
Mohali, Punjab, India
rankarachit5@gmail.com

Abstract—Artificial intelligence holds transformative potential for education in linguistically diverse nations, yet existing systems overwhelmingly depend on cloud infrastructure, operate primarily in English, and lack alignment with national curricula. This paper presents Shiksha Setu, a local-first AI tutoring platform whose ingestion pipeline addresses the complete National Council of Educational Research and Training (NCERT) catalog—559 textbooks spanning classes 1 through 12 in three publication languages—and which delivers retrieval-augmented instruction in ten Indian languages without transmitting student data beyond the local device. All results reported here are measured against an ingested corpus of 30 textbooks (9,722 indexed passages, classes 1–3 and 5–11); the pipeline is resumable and corpus size is a function of ingestion time rather than of capability. The system employs BGE-M3 multilingual embeddings with HNSW-indexed vector search over PostgreSQL, achieving cross-lingual retrieval cosine similarities of 0.596–0.672 on the successful three of five queries issued in four Indian languages against an English-language corpus. Through half-precision inference, the embedding pipeline reduces its memory footprint from 2,166 MB to 1,083 MB at a measured cosine fidelity of 0.999999 against full precision, enabling deployment on machines with as little as 4 GB of RAM—verified through kernel-level peak resident set size measurement rather than arithmetic estimation. The ingestion pipeline processes the 263-book taught curriculum at 140 seconds per book—a 1.38× speedup over sequential execution, established by re-ingesting an identical book set with prefetching disabled— and the platform discovers and addresses two classes of corpus corruption: legacy Devanagari fonts that render 66% of Hindi textbooks as Latin gibberish, and mathematical symbol erasure in PDF text layers. A security audit identifies and repairs twelve vulnerabilities, including a safety pipeline that silently failed open and a JWT implementation that accepted refresh tokens as access credentials. Reading support for dyslexic learners introduces akshara-level segmentation for nine Brahmic scripts, replacing the Latin-alphabet assumptions of conventional readability tooling. The system is validated by 445 automated tests with zero failures across unit, integration, and end-to-end suites.

Index Terms—Retrieval-Augmented Generation, Multilingual Education, Local-First AI, Indian Languages, Curriculum-Aligned Tutoring, Accessibility, NCERT

I. Introduction

India is home to 1.4 billion people speaking 22 constitutionally recognized languages, written in at least 13 distinct scripts. The National Council of Educational Research and Training (NCERT) publishes the curriculum that governs instruction for over 250 million school-age children, yet its textbooks exist only in English, Hindi, and Urdu. A Tamil-speaking student in rural Tamil Nadu, a Marathi-speaking student in Maharashtra, and a Bengali-speaking student in West Bengal all study the same curriculum—but none of them can read it in their mother tongue unless a human teacher translates it in real time.

Artificial intelligence can bridge this gap, but the dominant paradigm of cloud-hosted large language models introduces three structural failures for Indian education:

Language exclusion. The overwhelming majority of educational AI systems operate in English as their primary language. For the estimated 800 million Indians whose primary language is not English [1], this renders such systems inaccessible at the point of need.

Infrastructure dependence. Cloud-based solutions require consistent high-bandwidth internet connectivity and impose per-query costs through API billing. In Tier 2 and Tier 3 Indian cities and rural areas, this dependency makes sustained educational use economically impractical.

Pedagogical misalignment. General-purpose language models lack awareness of curriculum structure, grade-appropriate complexity, and the specific conceptual progression of NCERT instruction. A class 6 student asking about chemical reactions requires a fundamentally different explanation from a class 12 student on the same topic.

This paper presents Shiksha Setu, a platform that addresses all three failures through a measurement-driven engineering approach. Every claim is backed by an experiment whose procedure, data, and result are reported—including experiments that produced wrong results before being corrected. The contributions are:

1) An ingestion pipeline addressing all 559 NCERT textbooks, with automatic detection and rejection of legacy-font corruption that renders 27 of 41 Hindi books unreadable—discovered only through measurement.
2) A cross-lingual retrieval system enabling questions in ten Indian languages to retrieve relevant passages from an English corpus, with measured quality across four languages—three retrieving correctly at 0.596–0.672 cosine similarity, two queries failing—and diagnosis of the compound-term cause.

3) A memory optimization strategy verified through kernel-level measurement enabling the serving pipeline to operate within a 4 GB memory budget, correcting an earlier arithmetic estimate that understated consumption by 31%.
4) Akshara-level readability measurement for nine Brahmic scripts, replacing Latin-alphabet syllable counting with script-appropriate segmentation.
5) A security audit documenting twelve vulnerabilities—all repaired—including a safety pipeline that failed open due to an import error.
6) A grounding metric detecting when fluent, confident answers have drifted from source material—a failure mode indistinguishable from a correct answer without quantitative measurement.

II. Related Work

A. Retrieval-Augmented Generation

Lewis et al. [2] introduced retrieval-augmented generation (RAG) for knowledge-intensive NLP tasks, establishing the paradigm of conditioning language model output on retrieved passages. Shiksha Setu adopts this architecture with a critical modification: a grade-level filter inserted between retrieval and generation, because the same topic at class 6 and class 12 requires different explanations. Karpukhin et al. [3] demonstrated the superiority of dense passage retrieval over BM25. Ma et al. [4] showed that query rewriting before embedding improves retrieval quality—the technique resolving the romanized Hindi failure in Section V-B. Nogueira and Cho [5] established passage re-ranking with cross-encoder models; the BGE-Reranker component exists in the codebase but is not yet wired into the retrieval path.

B. Multilingual Embeddings and Indian Language NLP

Chen et al. [6] presented BGE-M3, a multilingual embedding model supporting over 100 languages in a unified 1024-dimensional vector space. Its cross-lingual property—semantically equivalent passages in different languages map to nearby vectors—allows a Hindi question to retrieve an English passage without explicit translation. Gala et al. [7] developed IndicTrans2 covering all 22 scheduled Indian languages, serving as the translation backbone. Kakwani et al. [1] provided monolingual corpora and evaluation benchmarks for Indian languages. Khanuja et al. [8] trained MuRIL on transliterated text, directly relevant to the romanized Hindi failure documented in Section V-B.

C. Output Language Drift and tRAG

Recent literature categorizes multilingual retrieval into frameworks such as MultiRAG (direct retrieval) and tRAG (translation-RAG). The platform described in this work formally implements a tRAG architecture [9], rewriting queries to a high-resource language (English) before retrieval. This architectural choice unknowingly resolves a newly documented vulnerability in large language models: Output Language Drift [10]. Studies show that when a multilingual RAG system processes a query in a target language but retrieves context in a dominant high-resource language, the model mixes languages and outputs the final answer in the high-resource language. The strict separation of the generation language from the delivery language via the translation layer elegantly neutralizes this cognitive drift.

D. Vector Search

The retrieval layer uses the HNSW graph structure proposed by Malkov and Yashunin [11], implemented through the pgvector extension for PostgreSQL. The index parameters ($m = 16$, $ef\_construction = 64$) balance build time against recall; measured search latency of approximately 20 ms for 12-nearest-neighbor queries confirms sub-interactive response times. Reimers and Gurevych [12] established the sentence-level embedding formulation used to load BGE-M3.

E. Reading Accessibility and Brahmic Scripts

Zorzi et al. [13] found that extra-large letter spacing improved reading speed and halved errors in dyslexic children—the single typographic intervention with strong experimental support and the default accessibility control in the platform. Rello and Baeza-Yates [14] and Kuster et al. [15] found no reliable advantage for dyslexia-specific typefaces. Nag [16] and Nag and Snowling [17] demonstrated that akshara-based literacy develops differently from alphabetic literacy, motivating the akshara-level segmentation described in Section VII.

F. Efficiency and Readability

Kincaid et al. [18] derived the Flesch-Kincaid grade-level formula defined on English syllable counts—which is why the platform refuses to compute it for Devanagari text. Xu et al. [19] demonstrated that generic simplification routinely loses content, motivating the measure-and-keep-the-easier approach. Micikevicius et al. [20] established the safety of half-precision arithmetic for neural network inference, providing the theoretical basis for the fp16 optimization that halves the BGE-M3 memory footprint.

III. System Architecture

A. Request Lifecycle

A student's question traverses an eight-stage pipeline from input to response. Table I summarizes the stages and measured latencies on an 8 GB Apple M1.

Without optional stages, a warm request completes in approximately three seconds. The design separates the question's language from the answer's language. A student may type in romanized Hindi and receive a response in Tamil. This independence is critical because NCERT publishes in only three languages while the platform serves ten. Translation is a serving-layer responsibility handled by IndicTrans2 [7].

TABLE I
Request Lifecycle Latency Benchmarks

Stage	Operation / Component	Latency
1	Rewrite to English / LLaMA-3.1-8B	∼0.6 s
2	Embed query / BGE-M3 (fp16)	∼0.1 s
3	Wide retrieval / pgvector HNSW	∼0.02 s
4	Class scoring / Sum-of-top-3	<0.01 s
5	Narrow retrieval / pgvector HNSW	∼0.02 s
6	Generate answer / LLaMA-3.1-8B	∼2.0 s
7	Reading support (opt.)	∼0.5 s
8	Illustration (opt.) / FLUX.1-dev	∼5.0 s

B. Model Stack

The platform integrates twenty-eight model identifiers, of which five are exercised on the critical path and measured in this work; the remainder are configured alternatives that this evaluation does not characterize. Table II separates the two, because a model stack presented as a single list would imply an empirical basis that only the first group has.

TABLE II
Model Stack. Upper block: measured on the critical path. Lower block: integrated, not evaluated here. † denotes a hosted model; all others are resident locally.

Role	Model	Params
Embeddings, grounding	BGE-M3	568M
Generation, rewriting	LLaMA-3.1-8B†	8B
Illustration	FLUX.1-dev†	12B
Vision OCR	Nemotron-Nano-12B†	12B
Generation (compared)	LLaMA-3.3-70B†	70B
Translation	IndicTrans2-1B	1B
Speech-to-Text	Whisper V3 Turbo [21]	809M
Text-to-Speech	MMS-TTS (11 ckpts)	∼100M
Curriculum Valid.	Gemma-2-2B-IT	2B
Text Simplification	Qwen2.5-3B	3B
Reranking	BGE-Reranker-v2-M3	568M

Exactly one model is resident locally on the critical path. This is not an accident of packaging but the property that makes the memory budget of Section VI achievable: embedding must be local because every chunk in the corpus passes through it during ingestion, whereas generation, illustration and OCR are per-request and therefore hosted, leaving no language-model weights resident during serving.

BGE-M3 is used twice for distinct purposes—retrieving passages, and later scoring how far an answer strayed from them (Section V-D). Both uses depend on the same property, cross-lingual embedding in a shared space, which is what permits a Devanagari passage and an English answer to be compared numerically at all.

Two integrated models are named but unusable in this deployment and are reported as such rather than omitted. The local OCR path (GOT-OCR2) requires CUDA and cannot execute on Apple silicon; its Python module additionally failed to import because torchvision was an undeclared dependency, which presented as a hardware limitation for some time before being diagnosed as a packaging one. The Tesseract fallback is installed without Devanagari training data, leaving it unable to read the script that most needs recovery (Section IV-C).

The choice of the 8B model over the 70B variant was driven by measurement (Table III).

TABLE III
Text Generation Model Latency Comparison

Model	Latency
meta/llama-3.1-8b-instruct	0.9 s
nvidia/nemotron-nano-9b-v2	2.5 s
meta/llama-3.3-70b-instruct	17–125 s
nvidia/llama-3.1-nemotron-nano-8b-v1	Timeout

C. Data Layer

The persistence layer combines PostgreSQL 17 with pgvector for HNSW-indexed similarity search, Redis 7 for response caching, and SQLite for the third cache tier. The multi-tier architecture (L1: in-memory LRU → L2: Redis → L3: SQLite) serves read-heavy endpoints. Cache keys incorporate a SHA-256 hash of the authorization header to prevent cross-user leakage. The vector index uses cosine distance:

```
CREATE INDEX idx_emb_hnsw_cosine
  ON embeddings USING hnsw
  (embedding vector_cosine_ops)
  WITH (m=16, ef_construction=64);
```

An earlier migration stored embeddings as double precision[] rather than vector(1024), making index creation impossible and condemning every search to a sequential scan.

D. Hardware Optimization

The platform implements five-phase optimization for Apple Silicon (Table IV).

TABLE IV
Apple Silicon Hardware Optimization Phases

Phase	Component	Improvement
1	Async-first I/O	19.4×
2	Type-opt. serialization	2.2 ms RT
3	GPU priority queue	0.3 μs sched.
4	P/E-core affinity routing	0.1 μs lookup
5	Buffer pool allocation	94.3% reuse

On Apple M4, measured benchmarks include GPU throughput of 3.72 TFLOPS (FP16), SIMD-accelerated cosine similarity at 54.7 million ops/sec, and full-stack import latency of 1.6 s.

IV. Corpus Construction

A. The NCERT Catalog

NCERT encodes every textbook with a five-character code—jesc1 represents Class 10, English medium, Science,

book 1. The subject letters are not algorithmically derivable: science is sc for classes 7–10 but cu (Curiosity) for class 6, and splits into ph, ch, bo at classes 11–12. The catalog is scraped from NCERT's textbook picker and cached locally.

The complete catalog contains 559 textbooks: 209 in English, 191 in Hindi, and 159 in Urdu. The platform teaches from a deliberate subset of 263 books—every English edition plus 54 subjects that exist only in Devanagari.

B. Ingestion Pipeline

The pipeline processes each book through five stages: ZIP download, per-chapter PDF extraction via PyMuPDF, text cleanup, semantic chunking (1200 characters, 200-character overlap, paragraph-boundary preference), BGE-M3 embedding (fp16), and storage in PostgreSQL. Each book is committed as a single database transaction for resumability. Download and embedding contend for no shared resource: one waits on a remote web server, the other saturates the GPU. Executed sequentially they idle in turn, so the next book's archive is fetched on a prefetch thread while the current book is embedded.

The speedup was established by a controlled comparison rather than inferred. The same three books (classes 6–8 Curiosity) were ingested twice, once with prefetching disabled through INGEST_PREFETCH=0, producing byte-identical output in both runs (37 chapters, 1,130 chunks): 193 s per book sequentially against 140 s per book with prefetching, a 1.38× speedup. Extrapolated, the 263-book taught curriculum requires approximately 10.2 hours on a single Apple M1.

We note that a naive comparison of figures taken under different conditions would have reported this backwards. An earlier sequential measurement of 97 s per book was recorded on a different, smaller book set; read against the 140 s figure it appears to show prefetching making ingestion slower. Only the paired measurement on identical inputs is informative, and unpaired throughput figures are excluded here for that reason.

C. Legacy Font Corruption

Two-thirds of the Hindi curriculum is rendered unreadable by every PDF text extractor tested. NCERT's older Hindi textbooks are typeset in Walkman-Chanakya 905, a legacy font mapping Devanagari glyphs onto ASCII codepoints with no ToUnicode table. Text extraction returns raw byte values. The analysis covers 41 of the 54 Devanagari-only books in the taught curriculum: the method samples a single chapter PDF per book, and for the remaining 13 books NCERT publishes no individual chapter files, returning HTTP 404 for chapters 1 through 3 at the documented URL scheme. Those 13 are absent for lack of a retrievable sample rather than by selection, and no result here is conditioned on their contents (Table V):

TABLE V
Hindi Corpus Corruption Analysis

Extraction Category	Books	Devanagari
Legacy font (Walkman-Chanakya)	27	0.0%
Unicode font (Kokila, Arya)	14	97.9–100%

The measurement that was inverted. An earlier quality assessment counted broken Devanagari sequences and reported the 27 legacy-font books as the cleanest in the corpus—a detector searching for damaged Devanagari finds nothing in a book containing no Devanagari at all. 133 chunks of Latin gibberish entered the retrieval corpus before the error was caught. The corrected detection identifies Hindi books whose extracted text is overwhelmingly Latin.

D. Text Layer Fidelity in Mathematics

PyMuPDF's text layer is lossy on mathematical content. Class 10 Mathematics chapter 4 defines a quadratic as $ax^2 + bx + c,\ a \neq 0$; the text layer yields "$ax2 + bx + c,\ a\ 0$"—the superscript flattened and the inequality deleted. Pages requiring OCR are identified by image density (Table VI).

TABLE VI
Image Density as OCR Predictor

Page Type	Images/1000 chars
Mathematics with equations	30.8
Science with figures	5.9
Prose with occasional figures	2.0–4.0

Cloud OCR (Nemotron VL) correctly recovers symbols but introduces substitution errors worse for learners: the text layer loses glyphs; the vision model substitutes plausible but incorrect characters and hallucinates nonexistent headers.

V. Cross-Lingual Retrieval

A. Evaluation Corpus

Every retrieval and grounding result in this section and the next was measured against a single corpus state, recorded here so that the numbers are interpretable and reproducible. The database held 30 textbooks—26 English and 4 Hindi—comprising 442 chapters and 9,722 indexed passages, with an embedding present for every passage (9,722 of 9,722; a passage lacking one is invisible to retrieval and is not otherwise detectable). Coverage spanned classes 1–3 and 5–11, with classes 4 and 12 absent.

This is a fraction of the 263-book taught curriculum, and the imbalance is load-bearing for two of the results rather than incidental to them. English science and mathematics books outnumbered the Hindi readers by roughly thirteen to one, which is what makes the grade-attribution result of Section V-D a test rather than a formality: a retriever that defaults to the majority of its corpus would answer every Hindi-literature question from

a class 10 science chapter. Where a result depends on corpus composition, that dependence is stated with it.

Absolute cosine similarities should be read as characteristic of this corpus size and not as a property of the method. Recall in particular is not estimated: with the curriculum partially ingested there is no ground-truth relevant set to recall against, and no such claim is made.

B. The Romanized Hindi Problem

BGE-M3's cross-lingual capability does not extend to romanized Hindi—the predominant input mode for Indian students using standard keyboards. The same chemistry question was embedded in three scripts (Table VII).

TABLE VII
Script Impact on Retrieval Quality

Script	Top Retrieval	Correct
English	Class 10 at 0.607	Yes
Devanagari	Class 10 at 0.564	Yes
Romanized Hindi	Class 1 at 0.520	No

Romanized Hindi drifts to class 1 picture books matching on surface-level token overlap. This failure provides a natural ablation study comparing naive direct retrieval (MultiRAG) against a translation-then-retrieval (tRAG) approach. Direct retrieval across scripts failed entirely. By rewriting every question into a short English search query before embedding (following Ma et al. [4]), the vector search is successfully forced back into the correct semantic neighborhood.

C. Cross-Lingual Retrieval Quality

Retrieval was evaluated with five queries spanning four Indian languages—Hindi, Marathi, Bengali and Tamil—against an English-only Class 10 Science corpus (Table VIII). Three queries retrieved the correct passage and two did not, so the four languages and the three successes describe different quantities and are reported separately throughout: coverage is four languages, and the similarity range of 0.596–0.672 characterizes only the three successful retrievals. Hindi appears twice because the same language succeeded on one topic and failed on another, which is the point of Section V-C: the failure is a property of the query term, not of the language.

TABLE VIII
Cross-Lingual Retrieval Results

Language	Topic	Sim.	Result
Hindi	Human eye focusing	0.672	Correct
Marathi	Electric current	0.637	Correct
Bengali	Heart function	0.596	Correct
Hindi	Photosynthesis	—	Failed
Tamil	Photosynthesis	—	Failed

The two failures share a common cause: compound scientific terminology. Both the Hindi and Tamil words for "photosynthesis" begin with the word for "light," causing retrieval to match the optics chapter rather than biology.

D. Grade-Level Classification

The platform uses the sum of each class's three strongest passage similarities as the classification signal. Two earlier functions were rejected: summing all similarities allowed volume to dominate (eight weak class 1 matches outscored three strong class 10 matches); averaging exhibited the opposite failure (a single high-similarity passage won by being its own mean). The vote_grade function lets each query independently propose a class on its own scale, weighted by decisiveness.

VI. Memory Optimization

A. Half-Precision Embeddings

BGE-M3's weights were evaluated at both full (float32) and half (float16) precision (Table IX).

TABLE IX
Precision Comparison for BGE-M3

Metric	float32	float16
Weight memory	2,166 MB	1,083 MB
Encode 4 queries	2,255 ms	1,197 ms
Model load time	8.5 s	8.1 s
Cosine fidelity	—	0.999999

Half precision yields 50% memory reduction and $\sim$1.88$\times$ speedup with no measurable quality degradation. A cosine similarity of 0.999999 cannot reorder any result list.

B. Serving Footprint: Measurement vs. Arithmetic

An earlier analysis summed component sizes to $\sim$1.6 GB and concluded comfortable fit within 4 GB. A dedicated benchmarking script measures actual peak RSS through getrusage(RUSAGE_SELF)—a high-water mark surviving page eviction, unlike ps which reported 46 MB for a process holding a 1 GB model (Table X).

TABLE X
Serving Pipeline Memory Footprint (Measured)

Pipeline Stage	Peak RSS
Interpreter start	14 MB
Configuration imported	265 MB
BGE-M3 loaded (fp16)	534 MB
Query encoded	2,094 MB
pgvector search complete	2,094 MB

The actual peak is 2,094 MB, not the 1,600 MB predicted. Loading model weights accounts for only 534 MB (safetensors memory-maps them); the first forward pass adds 1,560 MB as weight pages become resident and attention workspaces are allocated. The system fits within 4 GB with $\sim$1.4 GB usable headroom (accounting for $\sim$500 MB of operating system overhead), not the 2.4 GB arithmetic implied.

C. Ingestion Memory Management

A single process ingesting 263 books reached book 8 and entered an unrecoverable state: swap at 11.4 GB of 12.2 GB maximum, the process in uninterruptible I/O wait. SIGKILL is not delivered in that state. The solution is batch processing: one process per six books, with ∼20 seconds of model reloading per batch.

VII. Reading Support for Brahmic Script Readers

A. Akshara Segmentation

Dyslexia support tooling is overwhelmingly designed for English. Most Indian students read a Brahmic script where the fundamental unit is the akshara—a consonant cluster carrying an inherent or explicit vowel. The segmentation algorithm recognizes the virama (halant) in each Brahmic script as the signal that two consonants are conjoined, operating generically across Devanagari, Bengali, Gurmukhi, Gujarati, Odia, Tamil, Telugu, Kannada, and Malayalam.

Two details: (1) sentence counting must honor the danda, not only the Latin period; (2) Flesch-Kincaid is computed for English only—akshara counts are not syllable counts.

B. Measured Simplification Quality

The first simplification attempt produced text harder than the original (Table XI).

TABLE XI
Readability Measurement: Flesch-Kincaid Grade Level

Version	FK Grade
NCERT source text	7.0
First simplification attempt	8.2 (harder)
Measure-and-keep approach	6.9 (easier)

The corrected approach measures the original, rewrites with readability as sole objective, and retains whichever version is easier.

C. Evidence-Based Typography

Letter and word spacing (on by default) is the only intervention with strong evidence [13]. Dyslexia-specific fonts are offered as preferences but labeled as weakly evidenced [14], [15]. Nothing in the module diagnoses any condition; it adjusts presentation.

VIII. Security Analysis

A comprehensive audit identified twelve vulnerabilities, all repaired (Table XII).

Vulnerability #1 is the most consequential: the safety pipeline was designed as a three-pass content filter, but an import path error caused the entire pipeline to fail to load. The error was caught by a bare except clause returning (response, True), making the content safety system structurally incapable of blocking any response since deployment.

TABLE XII
Security Audit Findings (Selected)

#	Vulnerability / Consequence
1	Safety pipeline import error caught by bare except: every response marked safe without evaluation
2	JWT type claim unchecked: 7-day refresh tokens accepted as 30-minute access tokens
3	validate_required() never called: production booted with no JWT secret
5	Embedding column as double precision[]: no vector index possible
7	RLS compared NULL=NULL: shared curriculum invisible to all users
9	Sentry received student PII despite send_default_pii=False

IX. Grounding and Hallucination Detection

A retrieval-augmented system can retrieve the correct chapter and still generate a fabricated answer. When asked about a specific story from the Hindi curriculum, the system retrieved the correct chapter, then described events that never occurred. The class was correct, the chapter was correct, the language was correct, and every detail was invented.

The grounding score computes cosine similarity between the generated answer and the source passages. Lexical overlap cannot serve this purpose because the platform produces answers in a different language from source passages. BGE-M3's cross-lingual embedding property makes the metric functional in exactly this case. Calibrated against four hand-verified answers (Table XIII):

TABLE XIII
Grounding Score Calibration

Score	Verdict	Description
0.701	Correct	Poetry couplets from chapter
0.668	Correct	Poem quoting its own lines
0.583	Correct	Story events described accurately
0.489	Fabricated	Content bearing no relation to source

GROUNDING_MIN = 0.55 is set between the correct and incorrect answers. Below this threshold, the answer is regenerated with the detected drift named in the prompt.

X. Consolidated Results

A. Validation Suite

The platform is validated by 445 automated tests with zero failures: ∼350 unit, ∼60 integration, ∼20 end-to-end, and ∼15 performance benchmark tests.

B. System-Level Metrics

Table XIV summarizes key metrics.

TABLE XIV
Consolidated System Metrics

Metric	Value
NCERT catalog coverage	559 textbooks
Taught curriculum	263 textbooks
Supported languages	10
API routes	72
Ingestion throughput	140 s/book (1.38×)
Full curriculum time	∼7 hours
Peak serving memory (fp16)	2,094 MB
fp16 cosine fidelity	0.999999
Memory reduction (fp16)	50%
Encoding speedup (fp16)	1.88×
Response latency (warm)	∼3 s
Grounding threshold	0.55
Security vulns fixed	12
Automated tests passing	445/445

XI. Discussion

The measurement-driven methodology reveals a recurring pattern: intuitive engineering assumptions are frequently contradicted by empirical measurement, and the contradictions are more informative than the assumptions.

The legacy font detection illustrates this most starkly. A quality metric searching for damaged Devanagari ranked the most corrupted books as the best—because corruption manifested as the absence of Devanagari, which the metric interpreted as perfection. The memory estimation follows the same pattern: summing static sizes to 1.6 GB misses the 2.1 GB transient peak. The grounding score addresses a fundamental RAG limitation: a system can retrieve correctly, generate fluently, pass every surface check, and still fabricate content.

Limitations. (1) The grounding threshold is preliminarily calibrated on only four answers due to manual verification limits; this statistically small sample size requires expansion. (2) Cross-lingual retrieval is evaluated across five of ten supported languages. (3) 27 legacy-font Hindi books remain unprocessable. (4) Memory measurements have run-to-run variance of up to 414 MB.

XII. Conclusion and Future Work

This paper presented Shiksha Setu, a local-first AI tutoring platform whose pipeline addresses the complete NCERT catalog of 559 textbooks and which delivers instruction in ten Indian languages on consumer hardware with as little as 4 GB of RAM. The reported evaluation rests on 30 ingested textbooks and 9,722 indexed passages. Every performance claim has been substantiated through kernel-level measurement, and engineering failures have been documented alongside their corrections.

The principal contributions are: (1) corpus construction with legacy-font corruption detection affecting 66% of Hindi textbooks, (2) cross-lingual retrieval measured over five queries in four languages, three succeeding at 0.596–0.672 cosine similarity, (3) a grounding metric detecting hallucinated responses invisible to surface-level evaluation, (4) half-precision inference halving memory at 0.999999 cosine fidelity, (5) akshara-level readability for nine Brahmic scripts, and (6) a security audit repairing twelve production vulnerabilities.

Future work targets: conducting a human-in-the-loop pilot study with actual educators to evaluate pedagogical utility, expanding the hallucination annotation dataset (100+ response pairs) for robust grounding calibration, integrating a cross-encoder reranker to resolve compound-term failures (e.g., photosynthesis), implementing Entity Linking for polysemous term disambiguation, and developing a Walkman-Chanakya-to-Unicode transliterator.

References

[1] D. Kakwani et al., "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in Findings of EMNLP, 2020.

[2] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020.

[3] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in Proc. EMNLP, 2020, pp. 6769–6781.

[4] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting for retrieval-augmented large language models," in Proc. EMNLP, 2023.

[5] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv preprint arXiv:1901.04085, 2019.

[6] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," arXiv preprint arXiv:2402.03216, 2024.

[7] A. Gala et al., "IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages," Trans. Mach. Learn. Res. (TMLR), 2023.

[8] S. Khanuja et al., "MuRIL: Multilingual representations for Indian languages," 2021.

[9] Z. Xu et al., "Multilingual retrieval-augmented generation for knowledge-intensive task," arXiv preprint arXiv:2504.03616, 2025.

[10] Y. Chen et al., "Language drift in multilingual retrieval-augmented generation: Characterization and decoding-time mitigation," arXiv preprint arXiv:2511.09984, 2025.

[11] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, Apr. 2020.

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-Networks," in Proc. EMNLP, 2019, pp. 3982–3992.

[13] M. Zorzi et al., "Extra-large letter spacing improves reading in dyslexia," Proc. Nat. Acad. Sci. (PNAS), vol. 109, no. 28, pp. 11455–11459, 2012.

[14] L. Rello and R. Baeza-Yates, "Good fonts for dyslexia," in Proc. ACM SIGACCESS Conf. Comput. Accessibility (ASSETS), 2013.

[15] S. M. Kuster, M. van Weerdenburg, M. Gompel, and A. Bosman, "Dyslexie font does not benefit reading in children with or without dyslexia," Annals of Dyslexia, vol. 68, pp. 25–42, 2018.

[16] S. Nag, "Early reading in Kannada: The pace of acquisition of orthographic knowledge and phonemic awareness," J. Res. Reading, vol. 30, no. 1, pp. 7–22, 2007.

[17] S. Nag and M. J. Snowling, "Reading in an alphasyllabary: Implications for a language-universal theory of learning to read," Sci. Stud. Reading, vol. 16, no. 5, pp. 404–423, 2012.

[18] J. P. Kincaid, R. P. Fishburne, R. L. Rogers, and B. S. Chissom, "Derivation of new readability formulas for Navy enlisted personnel," Naval Tech. Training Command, Rep. 8-75, 1975.

[19] W. Xu, C. Callison-Burch, and C. Napoles, "Problems in current text simplification research: New data can help," Trans. Assoc. Comput. Linguist. (TACL), vol. 3, pp. 283–297, 2015.

[20] P. Micikevicius et al., "Mixed precision training," in Proc. ICLR, 2018.
[21] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proc. ICML, 2023.